Skill 的实战心法：让指令真正生效

- 来源: https://class.imooc.com/lesson/5288#mid=105145
- 阶段: 阶段五: 【能力封装】可复用技能
- 周: 第10周 Skill：可复用技能封装
- 模块: frontend-design实战
- 课节: 图文 1-3 Skill 的实战心法：让指令真正生效

1-3 Skill 的实战心法：让指令真正生效

Skill 的实战心法：让指令真正生效

> [!TIP] 阅读指南 前面六节我们走完了 Skill 的完整旅程：概念、生态、核心机制、扩展能力、创建方法、实战案例。该学的都学了。 但有一件事书上没明说，只有真正开始用 Skill 的人才会发现：有时候你在 SKILL.md 里写得很清楚，大模型就是不理你。不是偶尔不理你，是经常不理你。 这一节不教你怎么写 Skill，而是从根源上分析为什么会这样。你会发现这不是大模型「不听话」，而是 Skill 的工作机制决定了某些写法天生容易被忽略。我们会引入渐进加载、路由优化、上下文管理等核心概念，帮你把 Skill 从一个「撞大运」的功能变成一个可预期的工具。

先理解触发机制

先搞清楚一个最基本的问题：大模型什么情况下会读到 Skill 的内容?

每次新对话启动时，大模型只预读两个信息：每个已安装 Skill 的 name 和 description。就这两个字段，通常不到 100 个 token。完整的 SKILL.md 不会进入上下文。

只有当大模型判断当前任务和某个 Skill 的 description 匹配时，它才会主动去读那个 Skill 的 SKILL.md。读完以后，这一轮对话里 Skill 的指令生效。但下一轮新对话，一切清零，又得重新触发一遍。

这就是渐进加载的核心思想。Anthropic 的官方规范把它分成三个层次：

层级	内容	加载时机	大致体积
第一层	name + description	对话启动时全部加载	约 100 token
第二层	SKILL.md 正文	触发后加载	建议 < 5000 token
第三层	references/ scripts/ assets/	按需读取	不限

这套架构保证了即使装了几十个 Skill，也不会把对话启动时的上下文撑爆。但代价也很明显：description 是第一层，是第一道也是唯一一道关卡。如果 description 写得不准，后两层根本没有出场的机会。

这就是「写了但不生效」的第一个、也是最重要的根源。

description 是路由钥匙

description 承担了 Skill 的全部路由职责。这句话怎么强调都不过分，因为它是渐进加载架构中唯一被大模型预先扫描的信息。

很多人写 description 的习惯是往宽了写：「用于代码相关的任务」。这句话说了等于没说，因为大模型绝大部分任务都跟代码有关。面对这样一个描述，大模型有两种可能：要么在所有任务上都触发这个 Skill（等于没有路由），要么在所有任务上都不触发（大模型不确定它是否真的是最佳匹配）。

正确的做法是往窄了写，同时照顾两个方向：什么时候该用，什么时候不该用。

什么时候该用，什么时候不该用

对比两个 description：

```
不推荐：用于审查 SQL。

推荐：审查 PostgreSQL 数据库迁移脚本，检查前后向兼容性和回滚安全。
  仅当用户明确提到数据库迁移、Schema 变更或回滚计划时使用。
  不用于普通的 SQL 查询优化。
```

第二个描述里，大模型不仅能判断「这个任务匹配」，还能判断「这个任务不匹配」。「不匹配」的判断和前者的「匹配」一样重要，它防止了 Skill 在错误的任务上被错误触发。

官方规范里还提到一个常见的误区：

把 description 写成了功能介绍。例如「这个 Skill 提供了……它可以用来……」，这种第三人称的外围描述对大模型做路由决策帮助不大。更好的写法是直接以命令式口吻告诉大模型什么时候该用它，类似「Use this skill when……」或者「在处理迁移脚本时触发」。

描述不是写一次就完了

没有一个 description 第一次就完美。

有一个系统的测试方法：为 Skill 准备大约 20 条查询，其中大约一半应该触发这个 Skill，另一半不应该触发。然后逐条运行，观察大模型是否做了正确的路由决策。

如果一条本该触发的查询没有触发，说明 description 的描述过于狭窄，漏掉了这个场景。如果一条不该触发的查询反而触发了，说明描述不够精确，范围划得太宽。

每轮修改后对着这 20 条查询重跑一遍，观察触发率的变化。随着迭代，description 会越来越准。这不是可有可无的步骤，而是渐进加载架构下的必然要求：因为路由的准确度从一开始就不是大模型的事，而是你来控制的。

SKILL.md 的膨胀问题

假设 description 写得足够好，大模型正确触发了 Skill，读完了完整的 SKILL.md。现在问题来到了第二层：大模型读了，但它能抓到重点吗？

如果 SKILL.md 只有 30 行，核心指令一目了然。但如果 SKILL.md 达到了 200 行、300 行，大模型确实读完了，但能不能从几百行指令中准确识别出当前情景下最关键的那几行，就完全取决于大模型当时的注意力分配了。信息密度越高，核心指令被稀释的概率就越大。

渐进加载解决这个问题的思路是：不要试图把 SKILL.md 变薄，而是把不重要的东西移出去。

官方规范给出了一个明确的分层策略。SKILL.md 只放核心指令，也就是大模型触发后必须知道的信息。如果有一批参考知识或示例代码需要附带，放到 references/ 目录下，在 SKILL.md 里通过一句引用链接过去。大模型遇到相关情况时会主动去读，读不到也不影响核心流程的执行。

以 Anthropic 官方给出的 PDF Skill 为例。这个 Skill 需要处理 PDF 读取和表单填写两类场景，但表单填写很复杂，不是每次都需要。它的做法是在 SKILL.md 里只写 PDF 读取的核心流程，把表单填写指令单独放到 forms.md 里，在 SKILL.md 中用一句「如果需要填写表单，参见 forms.md」来引用。大模型读取 PDF 时不会额外消耗 token 去读表单指令，只有在真正遇到表单时才会主动打开 forms.md。

这种分层方式还有一层更深的含义：放在 references/ 里的内容几乎不受体积限制，因为大模型不会主动预读它们。渐进加载的本质不是限制内容总量，而是把内容按使用频率分层，高频的放 SKILL.md，低频的放 reference，需要执行的代码放 scripts/。

一个合理的 SKILL.md 应该控制在 200 行以内。超过这个长度，就应该考虑拆分了。拆分的信号包括：出现了「如果……就……」的条件分支（拆分到不同 reference 文件）、出现了大量示例代码（拆分到 scripts/）、出现了长篇的背景知识或概念解释（拆分到 references/）。

拆分后的大致结构是这样：

```
my-skill/                        ← Skill 目录
│
├── SKILL.md                     ← 触发后立即加载（~200 行核心指令）
│   ├── 核心流程说明
│   ├── 关键规则和约束
│   └── → 参见 references/custom-api.md
│       → 需要时执行 scripts/generate.sh
│
├── references/                  ← 按需读取，不占启动上下文
│   ├── custom-api.md            API 接口规范和常见参数
│   ├── error-handling.md        错误码对照表和处理建议
│   └── design-principles.md     设计原则和最佳实践
│
└── scripts/                     ← 按需执行，Agent 在需要时调用
    ├── validate.sh              环境校验脚本
    └── generate.sh              代码生成脚本
```

第一层是触发后立即装入的核心指令，控制在 200 行以内。第二层是 references/，大模型不会主动去读，但遇到具体问题（比如「这个 API 的参数是什么」）时会主动查阅。第三层是 scripts/，Agent 在确定需要执行时才调用。

装太多 Skill 也在稀释上下文

SKILL.md 内部的膨胀是一个问题，外部还有一个更隐蔽的问题：你装了多少个 Skill。

每次对话启动时，大模型会预读所有 Skill 的 name 和 description。每个 description 平均按 50-100 个 token 算，20 个 Skill 就是 1000-2000 个 token。这还不算大模型在「从这些描述中精准匹配当前任务」这个过程中消耗的注意力。

当 description 的列表越来越长，问题不只是在 context 中占用空间，更在于相似 description 之间的干扰。两个 Skill 的描述里都出现了「代码」「项目」「审查」这样的关键词，大模型选择错误的概率就会上升，或者在两个之间犹豫不决。

行业里的建议是把日常活跃的 Skill 控制在 8 个以内。那些偶尔才用到的 Skill，可以移出安装目录，放在一个独立的备份文件夹里。需要时再装回来，用完再移走。Skill 就只是一个文件夹，装和卸都是几秒钟的事。

控制数量不只是为了节省空间，更是为了让每个 Skill 的 description 在列表中足够显眼。描述之间的间距越大，路由就越清晰。

区分常驻规则和按需技能

渐进加载还有一个容易被忽视的推论：不是所有规则都适合放在 Skill 里。

不是所有规则都适合放在 Skill 里

Skill 是按需加载的，只有被触发时才生效。这意味着，如果有一些规则希望在每次对话中都起作用，比如「项目的代码风格是什么」「测试命令怎么运行」「commit message 用什么格式」，把这些规则放在 Skill 里，它只在触发后的那一轮生效，下一轮又得重新触发。

大多数编码助手都支持一个始终在线的上下文文件：Claude Code 的 CLAUDE.md、GitHub Copilot 的 .github/copilot-instructions.md、Cursor 的规则文件、Qoder 的 AGENTS.md。这个文件里的内容在每次对话开始时都会自动进入上下文，不需要触发，也不担心被遗忘。

我见过很多人把本应放在那个文件里的全局规则写进 SKILL.md，然后奇怪为什么有时候生效有时候不生效。通病在于没有区分「始终在线」和「按需加载」。全局规则放在全局配置里，专业技能放在 Skill 里，各管各的，这是第一道防线。

一个 Skill 只做一件事

这是一个在设计层面反复被验证的经验。

Skill 之间天然可以协作。一个项目里可以有「代码审查 Skill」和「测试生成 Skill」各自管好自己的领域，互不干扰。但很多人写 Skill 的习惯是越写越多，一个 Skill 里既管代码审查又管提交规范还管代码生成，希望一个搞定所有事情。

Anthropic 官方把 Skill 分为四种类型，可以对照着看自己的 Skill 属于哪一类：

类型	职责	典型场景
知识型	纠正大模型的默认知识	内部库的用法、反模式、团队约定
执行型	编排工具和脚本，形成工作流	代码生成、部署、迁移
验证型	检查结果是否正确	测试验证、UI 断言、安全检查
自动化型	处理周期性重复事务	周报生成、票据创建、依赖审计

如果一次处理的任务，比如审查代码、生成报告、部署上线，同时出现在同一个 SKILL.md 的条件分支里，那它至少跨了执行型和验证型两个类型，甚至有可能是四个类型的大杂烩。判断标准很简单：如果 SKILL.md 里出现了「如果用户需要 A，执行 B；如果用户需要 C，执行 D」这样的结构，就意味着它至少应该拆成两个独立的 Skill。各自有单独的 description，各自在合适的场景下被触发，互不污染对方的路由信号。

这不是对内容量的硬性限制，而是一个路由原则：每个 Skill 只拥有一个精确的、聚焦的描述空间。 描述越聚焦，触发越精准。当一个 Skill 试图覆盖多个场景时，它的 description 必然变得模糊，最终哪个场景都触发不好。

核心原则

以上五个方向最终可以抽象成一句话：Skill 的质量取决于触发，触发取决于路由，路由取决于 description 的精准度。 写 SKILL.md 是锦上添花，写 description 才是雪中送炭。

如果正在用 Skill 并且遇到了「不生效」的问题，不妨先检查 description 而不是 SKILL.md。先把路由讲清楚，再优化内容。渐进加载的全部智慧都建立在一条假设上：大模型能正确触发。如果 description 让它在第一步就错了，后面的一切都没有意义。

■ 学点英语

中文	English	音标	说明
触发率	Trigger Rate	/ˈtrɪgər reɪt/	衡量Skill被大模型正确触发的概率，描述精准度是核心影响因素
评估查询	Evaluation Queries	/ɪˌvæljuˈeɪʃn ˈkwɪəriz/	用于验证description路由准确性的测试查询集，包含应触发和不应触发两类用例
路由决策	Routing Decision	/ˈruːtɪŋ dɪˈsɪʒn/	大模型根据description和当前任务判断是否加载某个Skill的过程
上下文稀释	Context Dilution	/ˈkɒntekst daɪ ˈluːʃn/	过多Skill的description在启动时抢占上下文，降低路由精度的现象
分裂信号	Split Signal	/splɪt ˈsɪgnəl/	提示SKILL.md需要拆分的标志，如条件分支、大量示例代码或背景知识
技能分类	Skill Taxonomy	/skɪl tæk ˈsɒnəmi/	将Skill按职责分为知识型、执行型、验证型和自动化型的分类体系